

Registry Esterni e Integrazione con Azure Container Registry

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Classificare i tipi di registry container (pubblici, privati, on-premise) e descriverne le caratteristiche
- Creare Secret di tipo `kubernetes.io/dockerconfigjson` per autenticare il pull da registry privati
- Associare pull secret a ServiceAccount con `oc secrets link`
- Eseguire le principali operazioni su Azure Container Registry con i comandi `az acr`
- Configurare ACR con Managed Identity su ARO per eliminare credenziali statiche
- Creare robot account su Quay.io e integrarli con OpenShift
- Sincronizzare immagini tra registry con `oc image mirror` e `skopeo sync`
- Progettare l'architettura ACR + ARO con private endpoint nella stessa VNet

Teoria e Concetti Chiave

Tipologie di Registry Container

Tipo	Esempi	Caratteristiche
Pubblico	Docker Hub, Quay.io, GHCR (GitHub), MCR (Microsoft)	Accesso anonimo per pull; gratuito; rate limiting possibile
Privato cloud	Azure Container Registry (ACR), Amazon ECR, Google Artifact Registry	Accesso autenticato; integrazione nativa con cloud provider
Privato enterprise	Quay Enterprise (Red Hat), JFrog Artifactory, Harbor	Self-hosted o SaaS; policy avanzate; scanning integrato
Interno OpenShift	<code>image-registry.openshift-image-registry.svc:5000</code>	Integrato nel cluster; autenticazione OAuth OpenShift

Secret `kubernetes.io/dockerconfigjson`

Per fare pull da un registry privato, è necessario un Secret del tipo corretto:

```
kubernetes.io/dockerconfigjson
```

Contiene il file `.dockerconfigjson` con le credenziali nel formato Docker standard:

```
{
  "auths": {
    "myregistry.example.com": {
      "username": "myuser",
      "password": "mypassword",
      "auth": "<base64(myuser:mypassword)>"
    }
  }
}
```

Come associare il Secret ai Pod:

Metodo	Scope	Comando
Campo <code>spec.imagePullSecrets</code> nel Pod	Solo quel Pod	Aggiungere <code>name: <secret></code> nel manifest
Campo <code>imagePullSecrets</code> nel ServiceAccount	Tutti i Pod che usano quel SA	<code>oc secrets link <sa> <secret> --for=pull</code>
Pull secret globale in <code>openshift-config</code>	Tutti i nodi del cluster	<code>oc set data secret/pull-secret -n openshift-config</code>

Azure Container Registry (ACR): Panoramica

Azure Container Registry è il registry container gestito da Azure. Caratteristiche principali:

Feature	Descrizione
SKU Basic/Standard/Premium	Basic per dev, Premium per produzione (geo-replication, private endpoint)
Geo-replication	Replica le immagini in più region Azure (solo Premium)
Private Endpoint	Accesso privato via VNet senza attraversare Internet
Managed Identity	Autenticazione senza credenziali statiche tramite identità Azure
ACR Tasks	Build automatizzate di immagini container direttamente in Azure
Content Trust	Firma e verifica delle immagini (basato su cosign/Notary v2)
Vulnerability Scanning	Integrazione con Microsoft Defender per la scansione delle immagini

ACR con Managed Identity su ARO

La Managed Identity permette al cluster ARO di fare pull da ACR **senza password o token statici**. Il meccanismo si basa sull'identità del Service Principal del cluster ARO a cui viene assegnato il ruolo **AcrPull** sull'ACR.

Ruoli ACR:

Ruolo Azure	Permessi	Uso tipico
AcrPull	Pull di immagini	Cluster ARO, App Service
AcrPush	Push di immagini	CI/CD pipeline, build agent
AcrDelete	Delete immagini	Lifecycle management
AcrImageSigner	Firma immagini	Pipeline di sicurezza
Contributor	Tutti i permessi di gestione	Amministratori ACR

Quay.io: Robot Account

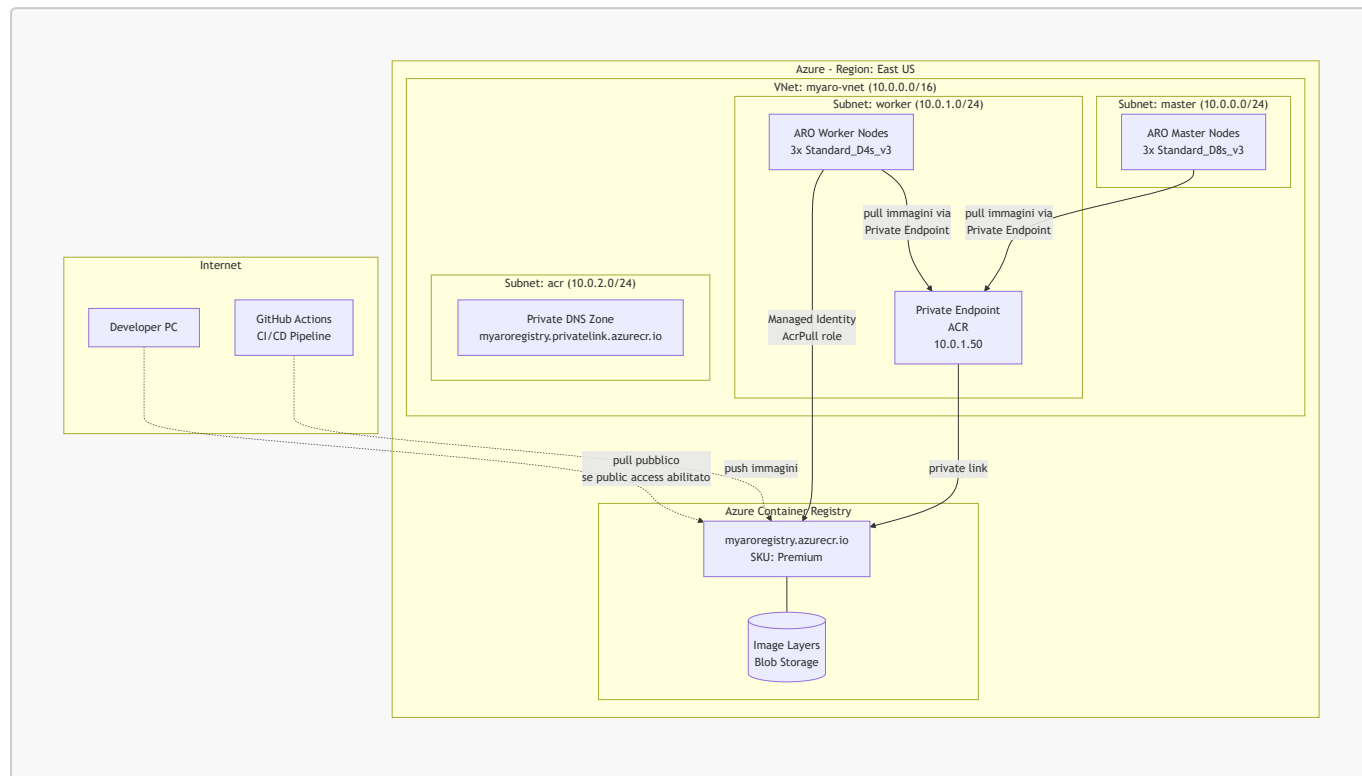
Quay.io (Red Hat) usa i **robot account** come account tecnici per l'autenticazione nelle pipeline CI/CD. Un robot account ha la forma **<organization>+<robot-name>** con un token generato automaticamente.

Mirror di Immagini: **oc image mirror** e **skopeo sync**

Strumento	Comando	Quando usarlo
oc image mirror	Copia singola immagine (o lista) tra registry	Mirror on-demand, operazioni occasionali
skopeo copy	Copia singola immagine	Flessibile, supporta autenticazioni diverse per src/dst
skopeo sync	Sincronizza un intero repository o lista da file YAML	Popolamento iniziale di un registry mirror
az acr import	Mirror server-side in ACR	Ambienti disconnessi, senza pull locale

Diagrammi

Architettura ACR Integrato con ARO (Private Endpoint + VNet)



Comandi Pratici con Output Atteso

```
# Creare un Secret dockerconfigjson per un registry privato generico
oc create secret docker-registry my-registry-secret \
  --docker-server=myregistry.example.com \
  --docker-username=myuser \
  --docker-password=mypassword \
  --docker-email=optional@example.com \
  -n myproject
```

```
# Output atteso:
# secret/my-registry-secret created
```

```
# Associare il pull secret al ServiceAccount "default" del namespace
oc secrets link default my-registry-secret --for=pull -n myproject
```

```
# Output atteso: (nessun output – comando silenzioso in caso di successo)
```

```
# Verificare che il SA abbia il secret associato
oc get sa default -n myproject -o jsonpath='{.imagePullSecrets}'
```

```
# Output atteso:
# [{"name":"my-registry-secret"}]
```

```
# Creare un nuovo Azure Container Registry (SKU Standard)
az acr create \
  --name myaroregistry \
  --resource-group myaro-rg \
  --sku Standard \
  --location eastus \
  --admin-enabled false          # ← disabilitare l'admin account; usare Managed
Identity
```

```
# Output atteso:
# {
#   "id": "/subscriptions/.../resourceGroups/myaro-
rg/providers/Microsoft.ContainerRegistry/registries/myaroregistry",
#   "loginServer": "myaroregistry.azurecr.io",
#   "name": "myaroregistry",
#   "sku": {"name": "Standard"},
#   "provisioningState": "Succeeded"
# }
```

```
# Elencare i repository presenti nell'ACR
az acr repository list \
```

```
--name myaroregistry \
--output table

# Output atteso:
# Result
# -----
# myapp
# ubi9/ubi-minimal
# ubi9/openjdk-21
```

```
# Elencare i tag di un repository specifico in ACR
az acr repository show-tags \
  --name myaroregistry \
  --repository myapp \
  --output table

# Output atteso:
# Result
# -----
# latest
# 1.0.0
# 1.1.0
```

```
# Assegnare il ruolo AcrPull al Service Principal del cluster ARO
ARO_SP=$(az aro show --name myaro --resource-group myaro-rg \
  --query servicePrincipalProfile.clientId -o tsv)
ACR_ID=$(az acr show --name myaroregistry --resource-group myaro-rg --query id -o
tsv)

az role assignment create \
  --assignee ${ARO_SP} \
  --scope ${ACR_ID} \
  --role AcrPull

# Output atteso:
# {
#   "principalId": "...",
#   "roleDefinitionName": "AcrPull",
#   "scope": "/subscriptions/.../resourceGroups/myaro-rg/.../myaroregistry",
#   ...
# }
```

```
# Mirror di un'immagine da registry.access.redhat.com a ACR con oc image mirror
oc image mirror \
  registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  myaroregistry.azurecr.io/ubi9/ubi-minimal:9.4 \
  -a /path/to/pull-secret.json
```

```
# Output atteso:
# phase 0:
# registry.access.redhat.com ubi9/ubi-minimal blobs=2 mounts=0 manifests=1
# info: Planning completed in 1.2s (2 blobs)
# uploading: myaroregistry.azurecr.io/ubi9/ubi-minimal sha256:a1b2c3... 29.41MiB
# info: Mirroring completed in 18.4s (1.6MB/s)
```

```
# Sincronizzare tutte le immagini da un file YAML di mapping con skopeo sync
cat > /tmp/mirror-list.yaml << EOF
registry.access.redhat.com:
  images:
    ubi9/ubi-minimal:
      - "9.4"
    ubi9/openjdk-21:
      - "1.19"
EOF
```

```
skopeo sync \
  --src yaml \
  --dest docker \
  /tmp/mirror-list.yaml \
  myaroregistry.azurecr.io
```

```
# Output atteso:
# Copying image registry.access.redhat.com/ubi9/ubi-minimal:9.4 to
myaroregistry.azurecr.io/ubi9/ubi-minimal:9.4
# Copying image registry.access.redhat.com/ubi9/openjdk-21:1.19 to
myaroregistry.azurecr.io/ubi9/openjdk-21:1.19
# Sync complete
```

```
# Creare un Secret per Quay.io con robot account
oc create secret docker-registry quay-robot-secret \
  --docker-server=quay.io \
  --docker-username=myorg+myrobot \
  --docker-password=<robot-token-from-quay> \
  -n myproject

# Associarlo al ServiceAccount default
oc secrets link default quay-robot-secret --for=pull -n myproject

# Output atteso:
# secret/quay-robot-secret created
```

Esempi YAML Completi

Secret dockerconfigjson + ServiceAccount + Deployment con immagine da ACR

```
# Secret di tipo dockerconfigjson per autenticarsi ad ACR
# NOTA: in produzione su ARO, preferire Managed Identity (no secret statici)
apiVersion: v1
kind: Secret
metadata:
  name: acr-pull-secret
  namespace: myproject
  labels:
    app: myapp
type: kubernetes.io/dockerconfigjson      # ← tipo obbligatorio per credenziali
registry
data:
  # Contenuto del file .dockerconfigjson codificato in base64
  # Struttura: {"auths":{"registry":
{"username":"...", "password":"...", "auth":"base64(user:pass)"}}}
  .dockerconfigjson:
eyJhdXRocyI6eyJteWFyb3JlZ2lzdHJ5LmF6dXJlY3IuawW8iOmsidXNlcm5hbWUiOiJteXVzZXIiLCJwYX
Nzd29yZCI6Im15cGFzc3dvcmQiLCJhdXRoIjoilYlhsMWMMyVnlpbTE1Y0dGemMzZHZjbVE9In19fQ==
---
# ServiceAccount dedicato per l'applicazione myapp
# Usa il pull secret ACR per scaricare l'immagine
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp-sa
  namespace: myproject
  labels:
    app: myapp
# Lista dei Secret da usare automaticamente per il pull delle immagini
imagePullSecrets:
- name: acr-pull-secret      # ← Secret con le credenziali ACR
---
# Deployment che usa l'immagine da ACR tramite il ServiceAccount myapp-sa
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: myproject
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
```



```
labels:
  app: myapp
spec:
  # Usa il ServiceAccount con il pull secret ACR
  serviceAccountName: myapp-sa      # ← SA con imagePullSecrets configurato

  containers:
  - name: myapp
    # Immagine da ACR – URL completo del registry privato
    image: myaroregistry.azurecr.io/myapp:1.0.0  # ← URL completo
    ACR/repo:tag

    # Pull policy: IfNotPresent per tag semantici (non :latest)
    imagePullPolicy: IfNotPresent      # ← usa la cache del nodo se
    disponibile

    ports:
    - containerPort: 8080
      name: http
      protocol: TCP

    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 512Mi
```

Note Specifiche per Azure ARO

Architettura ACR + ARO: Private Endpoint nello stesso VNet

Per ambienti di produzione ARO, la best practice è configurare ACR con un **Private Endpoint** nella stessa VNet del cluster, eliminando il traffico su Internet:

```
# Creare ACR Premium (richiesto per private endpoint)
az acr create \
  --name myaroregistry \
  --resource-group myaro-rg \
  --sku Premium \
  --public-network-enabled false          # ← disabilita accesso pubblico

# Creare il Private Endpoint per ACR nella subnet dei worker
az network private-endpoint create \
  --name acr-private-endpoint \
  --resource-group myaro-rg \
  --vnet-name myaro-vnet \
  --subnet worker-subnet \
  --private-connection-resource-id $(az acr show --name myaroregistry -g myaro-rg
--query id -o tsv) \
  --group-id registry \
  --connection-name acr-connection

# Creare la Private DNS Zone per la risoluzione del nome
az network private-dns zone create \
  --resource-group myaro-rg \
  --name privatelink.azurecr.io

az network private-dns link vnet create \
  --resource-group myaro-rg \
  --zone-name privatelink.azurecr.io \
  --name myaro-acr-dns-link \
  --virtual-network myaro-vnet \
  --registration-enabled false
```

Geo-replication ACR per Cluster ARO Multi-Region

Per cluster ARO distribuiti in più region Azure (disaster recovery), ACR Premium supporta la geo-replication:

```
# Aggiungere una replica ACR in West Europe (per un secondo cluster ARO)
az acr replication create \
  --name westeurope \
  --registry myaroregistry \
  --resource-group myaro-rg \
  --location westeurope

# Verificare le repliche configurate
az acr replication list \
  --registry myaroregistry \
  --resource-group myaro-rg \
  --output table

# Output atteso:
```

# Name	Location	ProvisioningState	Status
# -----	-----	-----	-----
# eastus	eastus	Succeeded	Ready
# westeurope	westeurope	Succeeded	Ready

ACR Tasks: Build Automatizzate di Immagini in Azure

ACR Tasks eseguono build di immagini container direttamente nell'infrastruttura Azure, senza richiedere un build agent locale:

```
# Eseguire una build rapida con ACR Tasks da un Dockerfile locale
az acr build \
  --registry myaroregistry \
  --image myapp:latest \
  --file Dockerfile \
  .

# Creare un'ACR Task automatica che si attiva al push su GitHub
az acr task create \
  --name myapp-build-task \
  --registry myaroregistry \
  --image myapp:{{.Run.ID}} \
  --context https://github.com/myorg/myapp.git \
  --file Dockerfile \
  --git-access-token <github-pat-token>
```

Riepilogo dei Punti Chiave

- I Secret di tipo `kubernetes.io/dockerconfigjson` contengono le credenziali per autenticarsi ai registry privati; possono essere associati a ServiceAccount con `oc secrets link`
 - ACR è il registry container managed di Azure; i livelli SKU sono Basic, Standard e Premium (Premium aggiunge private endpoint, geo-replication, content trust)
 - La configurazione **Managed Identity** (ruolo `AcrPull` sul Service Principal ARO) elimina la necessità di credenziali statiche per il pull da ACR
 - Per produzione ARO, configurare ACR con **Private Endpoint** nella stessa VNet per eliminare il traffico su Internet e ridurre la latenza
 - `oc image mirror` e `skopeo sync` permettono di copiare/sincronizzare immagini tra registry; `az acr import` esegue il mirror server-side direttamente in Azure
 - Quay.io usa i **robot account** come identità tecniche per CI/CD; si integrano con OpenShift tramite Secret `dockerconfigjson`
 - **ACR Tasks** permettono di eseguire build di immagini direttamente in Azure senza build agent locali
 - La geo-replication ACR (Premium) garantisce la disponibilità delle immagini anche in caso di interruzione di una region Azure
-

Domande di Verifica

1. Quale tipo di Secret Kubernetes contiene le credenziali per autenticarsi a un registry container privato?

- A) `kubernetes.io/basic-auth`
- B) `kubernetes.io/tls`
- C) `kubernetes.io/dockerconfigjson` ✓
- D) `kubernetes.io/service-account-token`

2. Quale comando associa un pull secret a un ServiceAccount in modo che tutti i Pod che usano quel SA possano fare pull dal registry privato?

- A) `oc set image sa/default --secret=my-secret`
- B) `oc secrets link default my-registry-secret --for=pull` ✓
- C) `oc patch sa/default --add-pull-secret=my-registry-secret`
- D) `oc create rolebinding pull-secret-binding --secret=my-registry-secret --sa=default`

3. Quale SKU di Azure Container Registry è richiesto per configurare un Private Endpoint e la geo-replication?

- A) Basic
- B) Standard
- C) Premium ✓
- D) Enterprise

4. Quale strumento permette di sincronizzare un intero repository di immagini da un registry verso un altro, usando un file YAML di mapping?

- A) `oc image mirror`
- B) `podman pull && podman push`
- C) `az acr import`
- D) `skopeo sync` ✓

5. Qual è il vantaggio principale di configurare la Managed Identity del cluster ARO con il ruolo `AcrPull` sull'ACR, rispetto all'uso di un Secret `dockerconfigjson`?

- A) La Managed Identity è più veloce nel pull delle immagini
- B) La Managed Identity non richiede credenziali statiche (username/password) da gestire, ruotare o rischiare di esporre ✓
- C) La Managed Identity bypassa il rate limiting di ACR
- D) La Managed Identity permette di fare anche il push delle immagini senza permessi aggiuntivi